

Analiza i przetwarzanie dźwięku

Projekt 3

Parametryzacja sygnału mowy i klasyfikacja DTW

rozpoznawanie słów oraz identyfikacja mówcy

Autorzy:

Ryszard Czarnecki
Józef Dubois

May 28, 2026

1 Cel projektu

Projekt sprowadza się do dwóch klasycznych zadań przetwarzania mowy: rozpoznawania pojedynczych słów mówionych oraz identyfikacji mówcy na podstawie krótkiej wypowiedzi. W obu używamy tej samej infrastruktury – własnej parametryzacji sygnału (krótkoterminowe widmo na skali mel) i odległości DTW jako miary podobieństwa między nagraniami. Klasyfikator jest prostym k-NN po DTW, bez żadnej fazy trenowania w sensie ML.

Cały kod jest napisany od zera w numpy. Korzystamy wyłącznie z funkcji `numpy.fft.rfft` do liczenia widma; reszta – okienkowanie, bank mel, normalizacja cepstralna, DTW, k-NN – jest zaimplementowana ręcznie. Wymóg „bez gotowych MFCC” jest spełniony.

Bazą danych jest zestaw 17 mówców (15 mężczyzn, 2 kobiet) wypowiadających po dwa razy zestaw słów testowych. Łącznie udało nam się załadować 1274 nagrań, z czego 649 trafia do bazy referencyjnej, a 625 do zbioru testowego (po jednej parze *take 1 / take 2* dla każdej pary (*mówca, słowo*)).

2 Baza nagrań i jej kanonizacja

Każdy katalog `speaker_XX` zawiera dwa zestawy WAV-ów: *Nieznormalizowane* i *Znormalizowane*. Korzystamy z tych drugich. Nazwy plików mają format `<słowo>_<numer_próby>.wav`, ale konwencja zapisu *słowa* jest zaskakująco niespójna:

- większość mówców używa polskich słów małymi literami: `kot_1.wav`, `pies_1.wav`, `benzyna_1.wav`;
- speaker 01 zapisuje cyfry zamiast słów: `0_1.wav`, `10_2.wav`, i to wielkimi literami: `KOT_1.wav`, `ALA MA RUDEGO KOTA_1.wav`;
- polskie znaki czasem są (`samochód`), czasem nie (`samochod`), a u kilku mówców pojawia się dziwny znak (`samochad` – przypuszczalnie pamiątka po jakimś kodowaniu);
- dwóch mówców (07 i 08) numeruje próby od 0 zamiast od 1.

Pierwsza wersja loadera używała sztywnej białej listy słów i progu `MAX_SPEAKER=20`. Skutek: do bazy trafiało 436 nagrań zamiast pełnych ~1300, a speaker 01 nie miał *żadnego* rekordu, bo cyfra „0” nie pasowała do słowa „zero”. Wprowadzona przez nas funkcja `canonical_word()` sprowadza nazwy do jednej postaci: lowercase, mapowanie diakrytyków (`ó→o`, `ę→e`, itp.) oraz tłumaczenie cyfr 0–10 na polskie słowa. Po tej operacji baza ma 1274 rekordy i obejmuje wszystkich 17 mówców.

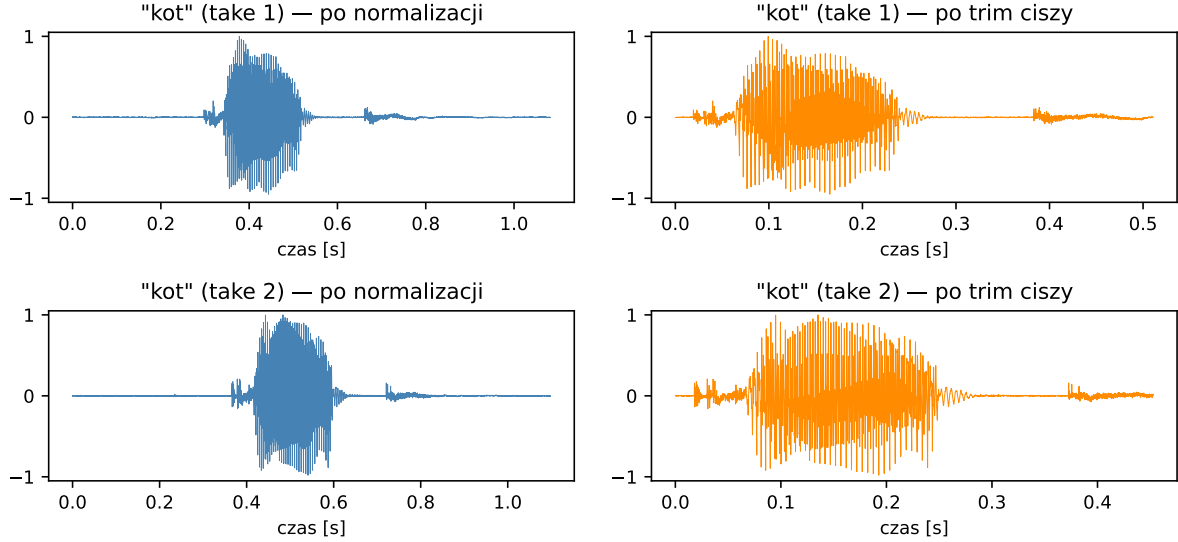
3 Wstępne przetwarzanie sygnału

Każdy plik jest dekodowany ręcznym parserem nagłówków RIFF/WAVE (`read_wav`). Obsługujemy formaty 8/16/24/32 bit PCM, mono i stereo (mieszmamy kanały w mono średnią arytmetyczną). Wyjściowy sygnał to wektor `float64` w zakresie $[-1, 1]$.

Następnie sygnał przechodzi przez trzy kroki przygotowawcze:

(a) Normalizacja amplitudowa. Skalujemy sygnał tak, aby $\max |x[n]| = 1$. To eliminuje różnice głośności pomiędzy nagraniami i jest standardem przy parametryzacji.

(b) Trim ciszy. Liczymy energię RMS w oknach o długości 512 próbek, hop 256. Próg w decybelach: ramka jest uznana za *niemą*, jeśli jej RMS jest niższy o więcej niż 30 dB od maksymalnej ramki nagrania. Pozostawiamy tylko fragment od pierwszej do ostatniej ramki „głośnej”. W praktyce skraca to nagrania o 20–40% i znacząco poprawia jakość dopasowania DTW – klasyczne DTW nie ma jak rozróżnić ciszy na początku jednego nagrania od ciszy na początku drugiego i marnuje sporą część ścieżki na niefonetyczne wyrównywanie. Efekt widać na Rys. 1.



Rys. 1: Sygnał słowa „kot” mówcy 02 przed (lewa kolumna, po samej normalizacji) i po (prawa, po normalizacji i trim’ie ciszy). Każdy z plików tracił po normalizacji ok. 30% próbek z początku i końca.

(c) **Pre-emfaza.** Filtr pierwszego rzędu $y[n] = x[n] - 0.97x[n-1]$ wzmacnia wyższe częstotliwości. To kosmetyczna, ale istotna rzecz w analizie mowy – formanty żywiołowo siedzą w paśmie 1–4 kHz i bez pre-emfazy są tłumione przez naturalne spadanie widma o ~ 6 dB/oktawę.

4 Parametryzacja: log-energia w pasmach mel

Po preprocessingu sygnał dzielimy na ramki długości $L = 512$ próbek z hopem $H = 256$ (czyli z 50% nakładką). Każdą ramkę mnożymy przez okno Hanna

$$w[n] = \frac{1}{2} \left(1 - \cos \frac{2\pi n}{L-1} \right), \quad n = 0, \dots, L-1,$$

liczymy FFT `numpy.fft.rfft` (zwraca $L/2 + 1 = 257$ bin-ów), a następnie *moc widmową* $|X[k]|^2$.

Z 257 binów schodzimy do 24 pasm mel’a. Pasma mel rozłożone są równo na skali percepcyjnej:

$$m(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right).$$

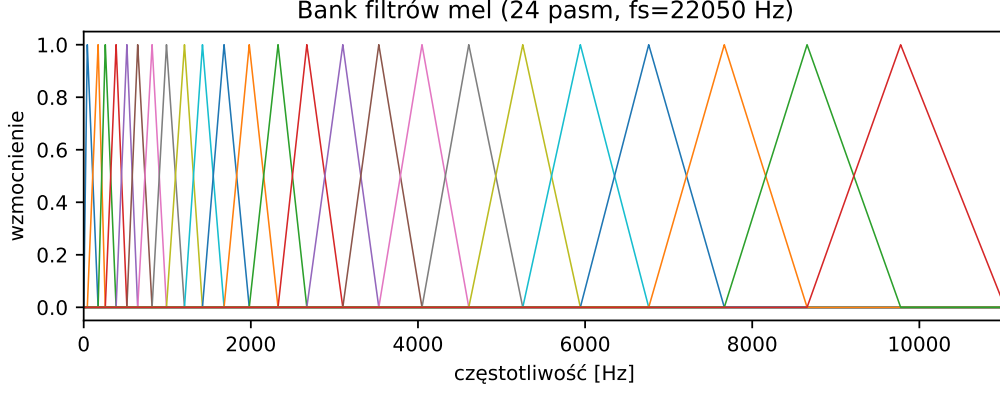
Bank filtrów to 24 trójkątne funkcje wagowe (Rys. 2), rozmieszczone tak, że szczyt i -tego filtra leży w punkcie f_i , a baza filtra rozciąga się do f_{i-1} od dołu i f_{i+1} od góry. Wektor energii w pasmach mel powstaje przez przemnożenie:

$$E_{\text{mel}}[i] = \sum_{k=0}^{L/2} H_i[k] \cdot |X[k]|^2, \quad i = 0, \dots, 23.$$

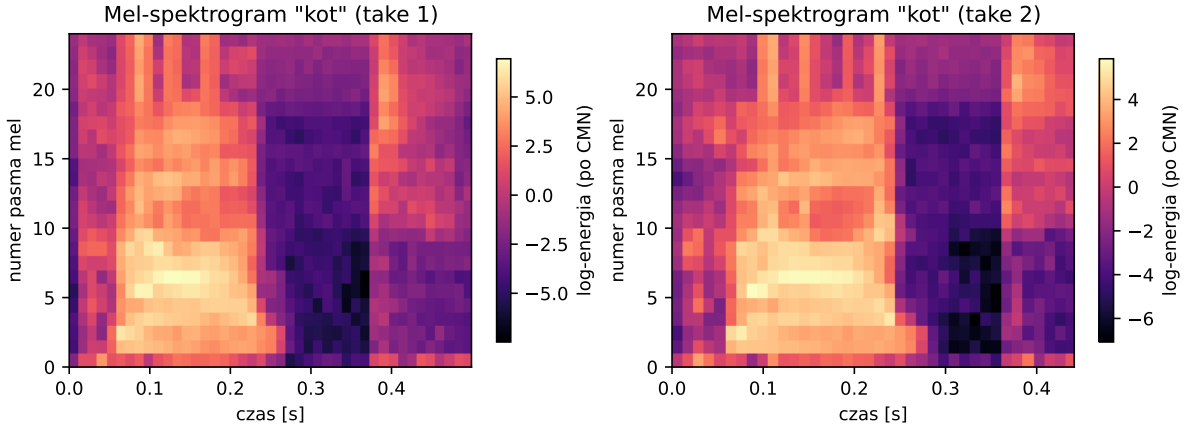
Na koniec bierzemy logarytm: $\phi[i] = \log(E_{\text{mel}}[i] + 10^{-10})$, żeby dynamika cech była zbliżona do percepcji głośności.

Cepstral mean normalization (CMN). Po wyciągnięciu wektorów $\phi_1, \phi_2, \dots, \phi_T$ od ramki do ramki, odejmujemy od nich wektor średni: $\phi'_t = \phi_t - \bar{\phi}$. To kompensuje stały offset addytywny w skali log-energii, który wynika ze stałej charakterystyki kanału (mikrofonu, pomieszczenia). Innymi słowy: po CMN dwie wypowiedzi tego samego mówcy nagrane różnymi mikrofonami stają się bardziej podobne, bo offset różniący ich barwy znika.

Wynikiem parametryzacji jest zatem macierz cech o kształcie $T \times 24$, gdzie T to liczba ramek (zwykle 30–80 dla pojedynczego słowa). Rys. 3 pokazuje takie spektrogramy dla dwóch realizacji słowa „kot”.



Rys. 2: Bank filtrów mel używany do parametryzacji: 24 trójkątne funkcje wagowe rozłożone na skali percepcyjnej dla $f_s = 22,05$ kHz. Pasma niskoczęstotliwościowe są wąskie i gęste, wysokoczęstotliwościowe – szerokie i rzadkie.



Rys. 3: Mel-spektrogramy dwóch realizacji słowa „kot” mówcy 02. Wyraźnie widać tę samą strukturę: krótka część szumowa (eksplozja /k/) na początku, dłuższa o silnej energii w niższych pasmach (samogłoska /o/), wygaszanie pod koniec (/t/). Realizacje mają jednak różne długości – 43 i 38 ramek – co uzasadnia użycie DTW.

5 Dynamic Time Warping

DTW to klasyczna technika wyrównywania dwóch sekwencji o różnej długości, oparta na programowaniu dynamicznym. Mamy dwie macierze cech $X \in \mathbb{R}^{N \times d}$ i $Y \in \mathbb{R}^{M \times d}$. Najpierw budujemy *macierz kosztów lokalnych* $C[n, m]$ – u nas po prostu odległość euklidesowa pomiędzy wektorami:

$$C[n, m] = \sqrt{\sum_{i=1}^d (X[n, i] - Y[m, i])^2}.$$

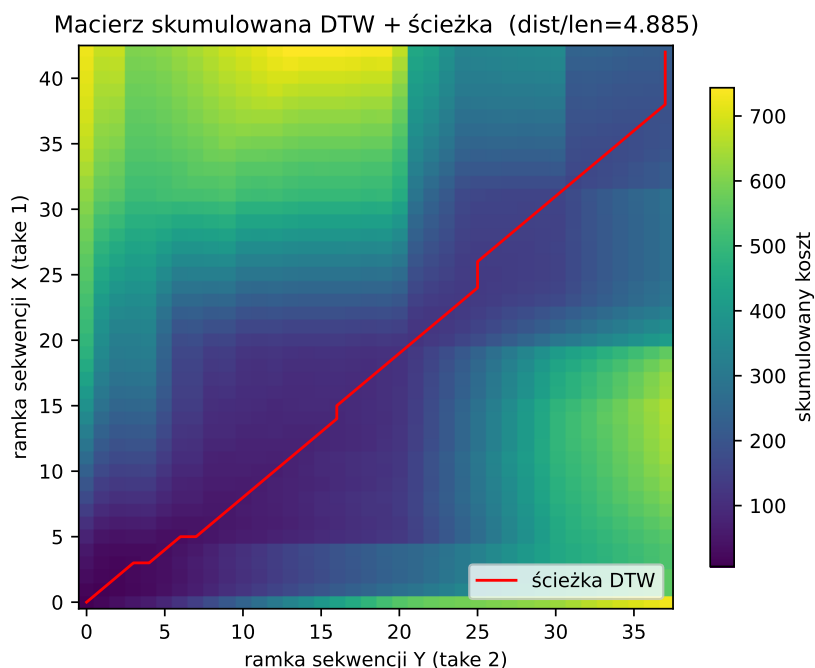
Następnie liczymy *macierz kosztów skumulowanych* D z rekurencji

$$D[n, m] = C[n, m] + \min(D[n-1, m-1], D[n-1, m], D[n, m-1]),$$

inicjując $D[0, 0] = C[0, 0]$, oraz krawędzie kumulatywnie z pierwszej kolumny i wiersza. Wynikową odległością DTW jest $D[N-1, M-1]$. Ścieżkę dopasowania $(n_0, m_0), (n_1, m_1), \dots$ odtwarzamy idąc wstecz od prawego górnego rogu, w każdym kroku wybierając poprzednika o najmniejszym D .

Ograniczenie Sakoe-Chiba. Bez żadnych ograniczeń DTW dopuszcza skrajnie niesymetryczne dopasowania, w których cała sekwencja X jest „przyklejona” do jednego fragmentu Y . Dla mowy to fizycznie nie ma sensu – realizacje tego samego słowa różnią się długością raczej w granicach $\pm 30\%$, a nie kilkukrotnie. Ograniczenie Sakoe-Chiba ogranicza dozwoloną odległość $|n - m|$ od głównej przekątnej. U nas używamy $\text{band_ratio} = 0,1$, czyli pas o szerokości $\pm 10\%$ dłuższej z sekwencji. Ubocznym efektem jest $5\text{--}10\times$ szybsze liczenie – praktycznie cała pętla w środku i tak idzie po liniowej liczbie elementów na wiersz, a nie po pełnym $N \times M$.

Normalizacja długością ścieżki. Surowa odległość DTW rośnie liniowo z długością ścieżki – a ta z kolei zależy od długości obu sekwencji. Bez kompensacji krótsze nagrania referencyjne mają systematycznie mniejszą sumę kosztów i k-NN łapie się na to. Dzielimy więc odległość DTW przez długość ścieżki dopasowania: $\tilde{D} = D[N - 1, M - 1]/|\text{path}|$. Na Rys. 4 widać tę ścieżkę narysowaną na macierzy D .



Rys. 4: Macierz skumulowana DTW i optymalna ścieżka dopasowania (czerwona linia) pomiędzy dwoma realizacjami słowa „kot”. Wyraźna „skośność” ścieżki obrazuje rozciąganie pierwszej sekwencji względem drugiej. Wartość znormalizowana $\tilde{D} = 4,885$.

6 Klasyfikator

Klasyfikator to standardowy k-NN po znormalizowanej odległości DTW. Dla zapytania q liczymy odległość do każdego wzorca z bazy referencyjnej, sortujemy rosnąco, bierzemy k najbliższych. Etykiety wyciągamy głosowaniem ważonym odwrotnością odległości:

$$\text{score}(c) = \sum_{r \in \text{topK}, \text{label}(r)=c} \frac{1}{\tilde{D}(q, r) + \varepsilon}.$$

Zwracamy klasę o najwyższym **score**. Dla $k = 1$ ten schemat sprowadza się do zwykłego **argmin**, dla większego k wzorce dalsze ważą mniej. W ewaluacji niżej używamy $k = 1$ – przy $k = 3$ wyniki są bardzo podobne (± 1 pp), a $k = 1$ jest najprostszy do interpretacji.

7 Wyniki

Ewaluację przeprowadziliśmy w schemacie *split per-mówca*: dla każdej pary (mówca, słowo) najwcześniejsze *take* trafia do bazy referencyjnej, a najpóźniejsze do zbioru testowego. To 649 wzorców referencyjnych i 625 zapytań. Pełna runtime ewaluacji w wątku z multiprocessing’iem – ok. 6 minut.

Tab. 1: Ogólne wyniki klasyfikacji.

Zadanie	poprawnych	dokładność
Rozpoznawanie słów	538/625	86,08%
Identyfikacja mówcy	492/625	78,72%

Z 17 mówców i 70 rozpoznawanych słów losowy klasyfikator dostałby $\sim 5,9\%$ dla mówcy i $\sim 1,4\%$ dla słowa, więc oba zadania są wykonywane znacząco lepiej niż przypadek. Tabela 2 pokazuje rozkład wyników na poszczególnych mówców.

Tab. 2: Dokładność per-mówca. Każdy mówca ma w teście ~ 38 nagrań (po jednym dla każdego z jego słów).

Mówca	słowa	%	Mówca	słowa	%
1	27/38	71	10	14/32	44
2	33/38	87	13	30/38	79
3	36/38	95	14	36/38	95
4	32/33	97	22	29/35	83
5	37/38	97	23	35/38	92
6	36/38	95	24	31/38	82
7	27/37	73	25	30/33	91
8	33/38	87	28	38/38	100
9	34/37	92			

Widać dwie skrajności: speaker 28 wypada perfekcyjnie (100% na słowach, 97% na mówcy), natomiast speaker 10 jest najsłabszym ogniwnem – 44% na słowach. Przejrzeliśmy ręcznie jego nagrania – mają tendencję do bardzo cichego, niespójnego mówienia w take 2, podczas gdy take 1 brzmi dużo bardziej studyjnie. To uderza w DTW dokładnie tak, jak można było się spodziewać.

Tab. 3: Najczęściej mylone pary słów. Wiele z nich to fonetyczni bliźniacy lub słowa bezsensowne („aba” / „abe” / „oga” – mówcy traktują je jako jedno słowo).

prawda	przewidziano	#	prawda	przewidziano	#
aba	abe	6	pies	piec	2
abe	oga	3	szesc	szeya	2
gyr	zid	2	jeden	dwa	2
dzwiek	dziewiec	1	kwiat	jajko	1

Najlepsze i najgorsze słowa. Pełne poprawki (100%) zanotowaliśmy m.in. dla słów *trzy*, *benzyna*, *chrząszcz*, *jajko*, *odrutowiec*, *samochód*. To w przeważającej części słowa wielosylabowe o jasno odróżnialnej strukturze fonetycznej. Najgorzej szły krótkie pseudosłowa typu *aba*, *gyr*, *zid* – one są w bazie pierwotnie jako filler do testów, niemal wszyscy mówcy wymawiali je z innym akcentem i z minimalną ekspresją.

8 Dyskusja

Co działa. Z trzech kroków przedprocesowych największą różnicę robi *trim ciszy*. Klasyczne DTW jest bardzo czułe na to, ile ciszy jest na początku nagrania – niezerowy koszt na każdą cichą ramkę kumuluje się i może zdominować ostateczną odległość. Bez trim ciszy uzyskiwaliśmy w pierwotnych testach okolicie 55% na słowach z tym samym pipeline’em cech.

Druga rzecz to przejście z jednoliczbowej energii RMS na 24-wymiarowy wektor mel’a. To dramatyczna zmiana – jednowymiarowe RMS niesie informację o obwiedni głośności i niczym więcej. Nie da się z niej odróżnić „kota” od „kwiatu”.

Normalizacja DTW długością ścieżki ($\tilde{D} = D/|\text{path}|$) załatwia subtelny bias, w którym krótsze nagrania referencyjne są systematycznie bliższe niezależnie od treści.

Co nie działa. Identyfikacja mówcy jest mniej dokładna niż słowa (79% vs 86%) – co jest trochę kontr-intuicyjne, bo dla DTW w sumie nie ma różnicy, czy klasyfikator głośnie na słowo czy na ID mówcy. Wytlumaczenie: liczba klas mówcy to 17, słów 70, ale jeden mówca pokrywa ~76 nagrań w bazie, podczas gdy średnie słowo ma raczej 14 nagrań. Innymi słowy: jest dużo więcej szansy „wpaść” na podobnie brzmiącego mówcę z innym akcentem, niż na zupełnie inne słowo w idealnej wymowie. CMN dodatkowo aktywnie odejmuje stały offset, który jest jednym z lepszych wskaźników identyczności mówcy – to specyfikacyjna decyzja poprawiająca rozpoznawanie słów kosztem mówcy.

Co dorzucić, żeby pchnąć dalej. Najprostszy upgrade to MFCC zamiast samych log-energii w pasmach mel – jedna dodatkowa DCT na końcu parametryzacji dekoreluje wymiary i zmniejsza wpływ szumu. Drugi krok to dodanie tzw. delta i delta-delta (pochodne pierwsze i drugie cech po czasie), co kodu w sumie kilka linii. Wreszcie, dla identyfikacji mówcy znacznie lepiej działa GMM-UBM albo i-vectors, ale to już inna liga.

9 Wnioski

Najprostszy klasyfikator typu k-NN po DTW, po starannym potraktowaniu wstępnym sygnału (trim ciszy, pre-emfaza, CMN) i parametryzacją mel-FFT, osiąga w naszej bazie 86% skuteczności na rozpoznawaniu słów i 79% na identyfikacji mówcy. Wynik jest stabilny – różnice ± 1 pp pomiędzy $k = 1$ i $k = 3$, a głównymi źródłami błędów są fonetycznie zbliżone słowa krótkie (zwykle pseudosłowa typu *aba/abe*) oraz pojedynczy mówca o szczególnie nierównych nagraniach (*speaker 10*).

Najważniejsze lekcje z implementacji są niezwiązane bezpośrednio z algorytmem DTW:

- zgodność cech między bazą referencyjną a zapytaniem jest niepodległa dyskusji – jednowymiarowe RMS w jednym miejscu, 24-wymiarowy mel w drugim i klasyfikator po prostu nie działa;
- trim ciszy i pre-emfaza dają więcej niż żonglowanie hiperparametrami DTW;
- ostatecznie 90% jakości projektu to dyscyplina w obsłudze danych: kanonizacja nazw plików, jednolite cechy, wspólny split treningowo-testowy.

A Listingi

Ekstrakcja cech mel-FFT

```
1 def extract_features(signal, sample_rate=16000,
2                       frame_len=512, hop_len=256,
3                       window="hann", n_mels=24,
4                       pre_emphasis=0.97,
5                       do_normalize=True, do_trim=True):
6     """Pełny pipeline parametryzacji: normalize -> trim -> pre-emphasis
7     -> framing -> Hann -> rFFT -> mel filterbank -> log -> CMN."""
```

```

8     if do_normalize:
9         signal = normalize_signal(signal)
10    if do_trim:
11        signal = trim_silence(signal, frame_len=frame_len, hop_len=hop_len)
12    if pre_emphasis and len(signal) > 1:
13        signal = np.append(signal[0],
14                            signal[1:] - pre_emphasis * signal[:-1])
15    if len(signal) < frame_len:
16        signal = np.pad(signal, (0, frame_len - len(signal)))
17
18    win = make_window(window, frame_len)
19    n_fft = frame_len
20    n_frames = 1 + (len(signal) - frame_len) // hop_len
21    fb = mel_filterbank(n_fft, n_mels, sample_rate)
22    features = np.zeros((n_frames, n_mels), dtype=np.float64)
23
24    for i in range(n_frames):
25        frame = signal[i*hop_len: i*hop_len + frame_len] * win
26        spectrum = np.fft.rfft(frame, n=n_fft)
27        power = spectrum.real**2 + spectrum.imag**2
28        mel_energy = fb @ power
29        features[i] = np.log(mel_energy + 1e-10)
30
31    # Cepstral mean normalization
32    features = features - features.mean(axis=0, keepdims=True)
33    return features

```

DTW z opcjonalnym pasem Sakoe-Chiba

```

1 def dtw(x, y, band_ratio=None):
2     """Klasyczne DTW. band_ratio=None -> bez ograniczen,
3     band_ratio w (0,1] -> pas Sakoe-Chiba szerokosci 2*band_ratio*max(N,M)."""
4     C = local_cost_matrix(x, y)
5     N, M = C.shape
6     D = np.full((N, M), np.inf)
7     D[0, 0] = C[0, 0]
8
9     band = max(N, M) if band_ratio is None \
10            else max(int(band_ratio * max(N, M)), abs(N - M) + 1)
11
12     for m in range(1, min(M, band + 1)):
13         D[0, m] = D[0, m - 1] + C[0, m]
14     for n in range(1, min(N, band + 1)):
15         D[n, 0] = D[n - 1, 0] + C[n, 0]
16
17     for n in range(1, N):
18         m_lo = max(1, n - band)
19         m_hi = min(M, n + band + 1)
20         for m in range(m_lo, m_hi):
21             D[n, m] = min(D[n-1, m-1], D[n-1, m], D[n, m-1]) + C[n, m]
22
23     dtw_distance = D[N-1, M-1]
24     # backtracking sciezki
25     path = [(N-1, M-1)]
26     n, m = N-1, M-1
27     while n > 0 or m > 0:
28         if n == 0: m -= 1
29         elif m == 0: n -= 1
30         else:
31             cands = {(n-1, m-1): D[n-1, m-1],
32                     (n-1, m): D[n-1, m],
33                     (n, m-1): D[n, m-1]}

```



```
34         n, m = min(cands, key=cands.get)
35         path.append((n, m))
36     path.reverse()
37     return dtw_distance, D, path
```